

L03 · Διαίρει & Κυρίευε I

Mergesort & Master Theorem

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Το πρώτο σχήμα σχεδίασης αλγορίθμων: διαίρει και κυρίευε. *Bubble/insertion sort, mergesort, τέσσερις διαφορετικές αποδείξεις του $O(n \log n)$, πύργος του Hanoi, και το Master Theorem.*

1 Τι θα δούμε

Αυτή είναι η **πρώτη** τεχνική σχεδίασης αλγορίθμων του μαθήματος. Στόχοι:

1. Να καταλάβουμε τι σημαίνει «**διαίρει και κυρίευε**» ως σχήμα — όχι ως ένας συγκεκριμένος αλγόριθμος, αλλά ως οικογένεια αλγορίθμων.
2. Να δούμε το πιο γνωστό παράδειγμα: τη **συγχωνευτική ταξινόμηση (mergesort)**, να αποδείξουμε ότι τρέχει σε $O(n \log n)$ με **τέσσερις διαφορετικούς τρόπους**, και να καταλάβουμε γιατί κάθε τρόπος είναι χρήσιμος.
3. Να βγάλουμε ένα γενικό εργαλείο — το **Master Theorem** — που λύνει ολόκληρη οικογένεια αναδρομικών σχέσεων χωρίς απόδειξη από την αρχή.
4. Να δούμε ότι D&C δεν είναι «μόνο για ταξινόμηση» — εφαρμόζεται και σε **πύργο του Hanoi**, και σε πολλά άλλα προβλήματα που θα δούμε στα επόμενα μαθήματα.

2 Σύντομη υπενθύμιση από το δεύτερο μάθημα

Ας συμφωνήσουμε στη γλώσσα πριν προχωρήσουμε. Από το δεύτερο μάθημα:

$$O(f(n)) = \{T : \exists n_0, \exists c > 0, \forall n \geq n_0, T(n) \leq c \cdot f(n)\}$$

$$\Omega(f(n)) = \{T : \exists n_0, \exists c > 0, \forall n \geq n_0, T(n) \geq c \cdot f(n)\}$$

$$\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

Και εναλλακτικά μέσω ορίων:

$\lim_{n \rightarrow \infty} f(n)/g(n)$	Συμβολισμός
$\alpha \neq 0$	$f \in \Theta(g)$
0	$f \in O(g), f \notin \Theta(g)$, δηλαδή $f \in o(g)$
∞	$g \in O(f), g \notin \Theta(f)$, δηλαδή $f \in \omega(g)$

Και η κλίμακα πολυπλοκότητων:

$$1 \prec \log n \prec n \prec n \log n \prec n^2 \prec n^3 \prec \dots \prec n^p \prec 2^n \prec 3^n \prec n!$$

Η πολυωνυμική πολυπλοκότητα είναι καλή· η εκθετική είναι κακή.

Κλειδί

Βέλτιστος αλγόριθμος (επανάληψη): αν για ένα πρόβλημα Π υπάρχει **κάτω φράγμα** $\Omega(f(n))$ — δηλαδή αποδεικνύεται ότι κάθε αλγόριθμος που λύνει το Π απαιτεί τουλάχιστον $\Omega(f(n))$ βήματα — και έχουμε αλγόριθμο σε $O(f(n))$, τότε ο αλγόριθμός μας είναι **ασυμπτωτικά βέλτιστος**. Στο τέλος αυτής της διάλεξης θα ξέρουμε ότι η mergesort είναι ακριβώς αυτή η περίπτωση.

3 Ζέσταμα: μέγιστο στοιχείο σε πίνακα

Πριν πάμε στο σύνθετο πρόβλημα της ταξινόμησης, ας δούμε ένα πολύ απλό για να εξασκηθούμε με την ανάλυση. **Πρόβλημα:** δοθέντος πίνακα $a_1, a_2, \dots, a_n$, να βρεθεί το μέγιστο στοιχείο.

Παράδειγμα

Αλγόριθμος 1 — «κράτα τον τρέχοντα μέγιστο».

```
v <- a_1
Για i = 2 έως n:
  Εάν v < a_i: v <- a_i
επίστρεψε v
```

Στοιχειώδης πράξη: σύγκριση. Κάνουμε **ακριβώς** $n - 1$ **συγκρίσεις**, ανεξάρτητα από την είσοδο. Άρα $\Theta(n)$.

Αλγόριθμος 2 — «ωθώ τον μέγιστο στο τέλος».

```
Για i = 1 έως n-1:
  Εάν a_i > a_{i+1}: ενάλλαξε
επίστρεψε a_n
```

Πάλι $n - 1$ συγκρίσεις. $\Theta(n)$.

Παρατήρηση. Και οι δύο αλγόριθμοι έχουν την ίδια ασυμπτωτική πολυπλοκότητα, αλλά ο 1ος είναι σαφώς προτιμότερος: ο 2ος *τροποποιεί* τον πίνακα εισόδου, ενώ ο 1ος όχι. Όταν δύο αλγόριθμοι έχουν το ίδιο ασυμπτωτικό κόστος, ψάχνουμε **άλλα κριτήρια** (μνήμη, ευκολία υλοποίησης, σταθερότητα, παράλληλη εκτέλεση).

4 Το πραγματικό πρόβλημα: ταξινόμηση

Πρόβλημα. Δοθέντος πίνακα από στοιχεία, να τα ταξινομήσουμε (με βάση κάποιο κριτήριο σύγκρισης).

4.1 Προφανείς εφαρμογές

- Οργάνωση μιας **playlist** ή αλφαβητική λίστα τηλεφώνων.

- Απαρίθμηση αρχείων σε έναν φάκελο αντίστροφα από την ημερομηνία.
- κ.ο.κ.

4.2 Λιγότερο προφανείς εφαρμογές

Μια ταξινομημένη λίστα είναι το «κλειδί» για πολλά:

- **Εύρεση μέσου στοιχείου.** Σε ταξινομημένη λίστα, το μεσαίο στοιχείο είναι σε $O(1)$.
- **Εύρεση κοντινού ζευγαριού.** Σε ταξινομημένη λίστα, το πιο κοντινό ζευγάρι αριθμών είναι κάποια διπλανή θέση — $O(n)$.
- **Δυαδική αναζήτηση.** Όπως είδαμε στο πρώτο μάθημα, σε ταξινομημένη λίστα η αναζήτηση είναι $O(\log n)$ αντί για $O(n)$.
- **Εντοπισμός ακραίων τιμών (outliers).** Στις άκρες της ταξινομημένης λίστας.
- **Εντοπισμός επαναλαμβανόμενων στοιχείων.** Διαδοχικά στοιχεία ή ίσα στοιχεία, σε $O(n)$.

Και **μη προφανείς** — όλες αυτές οι περιοχές χρησιμοποιούν ταξινόμηση ως υποπρόβλημα: συμπίεση δεδομένων (π.χ. Huffman coding, που θα δούμε αργότερα στους άπληστους αλγόριθμους), γραφικά H/Y (z-buffer ταξινόμηση πολυγώνων), υπολογιστική βιολογία (DNA alignment), και ελάχιστα συνδετικά δέντρα (αλγόριθμος Kruskal στην ενότητα των γραφημάτων — απαιτεί ταξινομημένες ακμές).

Διαίσθηση

Η ταξινόμηση είναι από τα **πιο μελετημένα** προβλήματα στην επιστήμη υπολογιστών γιατί είναι **παντού**. Ένας καλός αλγόριθμος ταξινόμησης ξεκλειδώνει βελτιώσεις σε όλα τα παραπάνω.

5 Πρώτη προσπάθεια: bubble sort & insertion sort

Πριν δούμε το «έξυπνο» mergesort, ας δούμε τα κλασικά απλά.

Παράδειγμα

Bubble sort — η «φυσαλίδα».

```

Διαδικασία bubblesort(a_1, ..., a_n):
  Για i = 1 έως n-1:
    Για j = 1 έως n-i:
      Αν a_j > a_{j+1}: εναλλάξε a_j <-> a_{j+1}

```

Διαίσθηση. Σε κάθε πέρασμα του εξωτερικού βρόχου, το «μεγαλύτερο» στοιχείο «αναδύεται» (σαν φυσαλίδα) στο σωστό του τέλος. Μετά από $n - 1$ περάσματα, όλα είναι στη θέση τους.

Πολυπλοκότητα. Διπλά φωλιασμένος βρόχος. Στο πρώτο πέρασμα κάνουμε $n - 1$ συγκρίσεις, στο δεύτερο $n - 2$, ..., στο τελευταίο 1 σύγκριση. Σύνολο:

$$(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2} = \Theta(n^2)$$

Στη **χείριστη περίπτωση** (πίνακας αντίστροφα ταξινομημένος) γίνονται όλες οι ανταλλαγές — $\Theta(n^2)$.

Παράδειγμα

Insertion sort — η «ενθετική».

```
Διαδικασία insertionsort(a_1, ..., a_n):  
  Για i = 1 έως n:  
    v <- a_i  
    j <- i - 1  
    Όσο j >= 1 και v < a_j:  
      a_{j+1} <- a_j      % ολίσθηση  
      j <- j - 1  
    a_{j+1} <- v
```

Διαίσθηση. Όπως ταξινομούμε τα φύλλα ενός παιχνιδιού καρτών στο χέρι: παίρνουμε κάθε νέο φύλλο και το βάζουμε στη σωστή θέση *μεταξύ* των ήδη ταξινομημένων.

Πολυπλοκότητα. Στη χειρίστη περίπτωση (αντίστροφα ταξινομημένος πίνακας) πάλι $\Theta(n^2)$ — κάθε νέο στοιχείο πρέπει να «κατέβει» σε όλο τον ήδη ταξινομημένο πρόθεμα.

Παρατήρηση. Στη **βέλτιστη** περίπτωση (ήδη ταξινομημένος πίνακας) η insertion sort είναι $\Theta(n)$ — κάθε νέο στοιχείο μένει στη θέση του. Η bubble sort *χωρίς early-exit optimization* παραμένει $\Theta(n^2)$ ακόμα και στην καλύτερη περίπτωση.

Προσοχή

Και οι δύο αλγόριθμοι έχουν **χειρίστη πολυπλοκότητα** $\Theta(n^2)$. Για $n = 10^6$ (ένα εκατομμύριο στοιχεία), αυτό σημαίνει 10^{12} πράξεις — χρόνος μιας **ώρας** σε σύγχρονο επεξεργαστή. Η mergesort που θα δούμε αμέσως κάνει την ίδια δουλειά σε $20 \cdot 10^6 \approx 2 \cdot 10^7$ πράξεις — **κάποια δευτερόλεπτα**. Η διαφορά n^2 vs $n \log n$ είναι **πέντε τάξεις μεγέθους** εκεί.

6 Η ιδέα: συγκρίσεις ανά δύο, μετά συγχώνευση ανά δύο

Πριν τυποποιήσουμε, η διαίσθηση: **σπάμε** τον πίνακα σε ζευγάρια, ταξινομούμε κάθε ζευγάρι (τετριμμένο), και μετά **συγχωνεύουμε** ζευγάρια σε τετράδες, τετράδες σε οκτάδες, και ούτω καθεξής μέχρι να έχουμε έναν ταξινομημένο πίνακα.

Divide et impera!

(λατινικό: «διαίρει και βασίλευε»)

7 Διαίρει και Κυρίευε — το γενικό σχήμα

Η τεχνική **διαίρει και κυρίευε (D&C)** βασίζεται σε τρία βήματα:

1. **Διαιρούμε** τα δεδομένα του προβλήματός μας σε μικρότερα υποπροβλήματα που **δεν έχουν κοινά στοιχεία** (ξένα μεταξύ τους).
2. **Κυριεύουμε** τα μικρότερα υποπροβλήματα **ξεχωριστά** (και **αναδρομικά** — εφαρμόζοντας ξανά τον ίδιο αλγόριθμο).
3. **Συνδυάζουμε** τις λύσεις που βρήκαμε για να λύσουμε το αρχικό πρόβλημα.

7.1 Πώς συνήθως εφαρμόζεται

Στη συνηθισμένη μορφή:

- **Διαιρούμε** δεδομένα μεγέθους n σε **δύο** μικρότερα στιγμιότυπα μεγέθους $\lceil n/2 \rceil$ και $\lfloor n/2 \rfloor$ (ξένα μεταξύ τους).
- **Επιλύουμε** καθένα με **αναδρομική κλήση** του ίδιου αλγορίθμου. Θεωρούμε την αναδρομική κλήση «**μαύρο κουτί**» που μας επιστρέφει τη λύση.
- **Συνδυάζουμε** τις δύο επιμέρους λύσεις σε λύση του πλήρους προβλήματος.

7.2 Γιατί δουλεύει: επαγωγική κατασκευή

Η τεχνική D&C βασίζεται σε **επαγωγική κατασκευή της λύσης**. Αρκεί (και χρειάζεται) να ξέρουμε:

1. **Βάση επαγωγής:** να επιλύσουμε το πρόβλημα για **μικρή (σταθερή) διάσταση δεδομένων** — π.χ. για $n = 1$ ή $n = 2$.
2. **Επαγωγικό βήμα:** να **συνδυάσουμε** τις ορθές λύσεις των ξένων υποπροβλημάτων μας σε **μία καθολική λύση** για όλα τα δεδομένα.

Κλειδί

Η σύνδεση με μαθηματική επαγωγή είναι ακριβής. Αν αποδείξεις (α) ότι ο αλγόριθμος δουλεύει για $n = 1$ και (β) ότι, υποθέτοντας ότι δουλεύει για κάθε $k < n$, δουλεύει και για n — τότε **έχεις απόδειξη ορθότητας**. Αυτό είναι το πρότυπο απόδειξης ορθότητας για κάθε D&C αλγόριθμο.

8 Συγχωνευτική ταξινόμηση — η ιδέα

Πώς ταξινομούμε σε αύξουσα σειρά με D&C;

1. **Βάση.** Αν ο πίνακας έχει **ένα στοιχείο**, είναι ήδη ταξινομημένος. Επιστρέφουμε τον ίδιο πίνακα.
2. **Αλλιώς:** **διαιρούμε** τον πίνακα **στη μέση** (δύο ίσα μισά): **ταξινομούμε** καθένα από τα δύο με **μία αναδρομική κλήση**. **συγχωνεύουμε** τις δύο ταξινομημένες λίστες σε μία.

Η ιδέα είναι: «αν ξέρω να συγχωνεύσω **δύο ταξινομημένες λίστες**, και αν ξέρω να σπάσω έναν πίνακα στη μέση, τότε μπορώ να ταξινομήσω τα πάντα» — η αναδρομή κάνει το υπόλοιπο.

9 Επεξεργασμένο παράδειγμα: ταξινόμηση 11 στοιχείων

Ακολουθούμε τον αλγόριθμο στον πίνακα $[10, 5, 8, 6, 3, 4, 7, 1, 9, 2, 11]$:

1. **Είσοδος:** $[10, 5, 8, 6, 3, 4, 7, 1, 9, 2, 11]$.
2. **Διαιρεί στη μέση:** αριστερό μισό $[10, 5, 8, 6, 3, 4]$ (6 στοιχεία), δεξί μισό $[7, 1, 9, 2, 11]$ (5 στοιχεία).
3. **Αναδρομικά,** κάθε μισό επιστρέφει ταξινομημένο: αριστερό $\rightarrow [3, 4, 5, 6, 8, 10]$, δεξί $\rightarrow [1, 2, 7, 9, 11]$.
4. **Συγχώνευση** των δύο $\rightarrow [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]$.

Το σύνολο της δουλειάς ξοδεύτηκε στο βήμα 4, που είναι γραμμικό. Στην πραγματικότητα η αναδρομή πάει βαθύτερα — το αριστερό μισό σπάει σε δύο, καθένα από αυτά σπάει σε δύο, ..., μέχρι να φτάσουμε σε πίνακες μήκους 1.

10 Η συγχώνευση δύο ταξινομημένων πινάκων

Πριν αποδείξουμε ότι η mergesort είναι $O(n \log n)$, χρειαζόμαστε την «βασική πράξη» που μετατρέπει τα ταξινομημένα μισά σε ταξινομημένο σύνολο: τη **συγχώνευση**.

Παράδειγμα

Πρόβλημα. Έστω a και b δύο **ήδη ταξινομημένοι** πίνακες με m και n στοιχεία αντίστοιχα. Να παράξουμε έναν ταξινομημένο πίνακα c με $m + n$ στοιχεία που περιέχει όλα τα στοιχεία.

Αλγόριθμος.

Διαδικασία συγχώνευση(a, b):

$i \leftarrow 1$

$j \leftarrow 1$

Για $k = 1$ έως $m + n$:

Εάν $a_i < b_j$:

$c_k \leftarrow a_i$

$i \leftarrow i + 1$

Αλλιώς:

$c_k \leftarrow b_j$

$j \leftarrow j + 1$

(Με κατάλληλους ελέγχους ορίων όταν εξαντληθεί ένας από τους δύο πίνακες — αντίγραψε ό,τι μένει από τον άλλον.)

Διαίσθηση. Δύο δείκτες (i, j) σαρώνουν τους δύο πίνακες ξεκινώντας από την αρχή. Σε κάθε βήμα συγκρίνουμε τα κορυφαία στοιχεία και τραβάμε το μικρότερο στο αποτέλεσμα. Σαν δύο σειρές στο supermarket να γίνονται μία.

Πολυπλοκότητα. Κάθε βήμα του βρόχου είναι σταθερού κόστους (μία σύγκριση + μία ανάθεση + μία αύξηση δείκτη). Ο βρόχος εκτελείται $m + n$ φορές $\rightarrow O(m + n)$, δηλαδή γραμμικός στο σύνολο των στοιχείων.

Χρόνος μνήμης. Χρειάζεται ένας επιπλέον πίνακας μεγέθους $m + n$ — η συγχώνευση **δεν** γίνεται in-place στην απλή της εκδοχή.

11 Ο αλγόριθμος mergesort

Με τη συγχώνευση στο χέρι, ο πλήρης αλγόριθμος είναι σχεδόν τετριμμένος:

Παράδειγμα

Mergesort (αναδρομική).

Διαδικασία mergesort(A, p, r):

Εάν $p = r$:

επίστρεψε $A[p..r]$ // βάση: ένα στοιχείο, ήδη ταξινομημένο

Εάν $p < r$:

$q \leftarrow \text{floor}((p + r) / 2)$ // σπάμε στη μέση

mergesort(A, p, q) // ταξινομούμε αναδρομικά το αριστερό μισό

mergesort(A, q + 1, r) // ταξινομούμε αναδρομικά το δεξί μισό

επίστρεψε συγχώνευση($A[p..q]$, $A[q+1..r]$)

Παρατήρηση. Καλούμε αρχικά mergesort(A, 1, n).

Γιατί στη μέση; Θα μπορούσαμε να διαιρέσουμε σε « $n - 2 + 2$ » αντί για « $n/2 + n/2$ »; **Όχι** — η πολυπλοκότητα τότε εκφυλίζεται. Θα το δούμε στην ανάλυση παρακάτω.

12 Το δέντρο αναδρομής

Πριν αποδείξουμε ότι $T(n) = O(n \log n)$, ας δούμε **οπτικά** πού πάει η δουλειά. Για $n = 8$, το δέντρο αναδρομής έχει τα εξής επίπεδα:

Επίπεδο	Πλήθος υποπροβλημάτων	Μέγεθος καθενός	Δουλειά επιπέδου
0 (ρίζα)	1	$n = 8$	8
1	2	4	$2 \times 4 = 8$
2	4	2	$4 \times 2 = 8$
3 (φύλλα)	8	1	$8 \times 1 = 8$

Παρατήρησε το κλειδί: **κάθε επίπεδο** του δέντρου κάνει **συνολικά n συγκρίσεις**. Το γιατί;

- Στο επίπεδο 1 έχουμε 2 υποπροβλήματα μεγέθους $n/2$. Κάθε ένα κοστίζει $n/2$ για συγχώνευση $\rightarrow$ σύνολο n .
- Στο επίπεδο 2 έχουμε 4 υποπροβλήματα μεγέθους $n/4$. Κάθε ένα κοστίζει $n/4 \rightarrow$ σύνολο n .
- Σε γενικό επίπεδο k έχουμε 2^k υποπροβλήματα μεγέθους $n/2^k$. Σύνολο n .

Και υπάρχουν $\log_2 n$ **επίπεδα** πριν φτάσουμε σε πίνακες μεγέθους 1. Άρα $T(n) = n \cdot \log_2 n$. Αυτή είναι η διαίσθηση — τώρα οι τυπικές αποδείξεις.

13 Πολυπλοκότητα mergesort — αναδρομική σχέση

Ορίζουμε $T(n)$ ως την πολυπλοκότητα του αλγορίθμου για είσοδο μεγέθους n :

$$T(n) = \begin{cases} 0 & \text{εάν } n = 1 \\ T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n & \text{διαφορετικά} \end{cases}$$

Σε λόγια: για $n = 1$ ο πίνακας έχει ένα στοιχείο, καμία πράξη· για $n > 1$ — κόστος για το αριστερό μισό + κόστος για το δεξί μισό + n για τη συγχώνευση.

Στόχος: να δείξουμε ότι $T(n) = O(n \log n)$. Θα παρουσιάσουμε **τέσσερις διαφορετικές αποδείξεις** για να εξοικειωθούμε με διάφορες τεχνικές χειρισμού αναδρομικών σχέσεων. Στην εξέταση, ξέρε **τουλάχιστον δύο** από αυτές.

14 Απόδειξη #1 — απλοποίηση της σχέσης (όταν $n = 2^k$)

Υποθέτουμε ότι το n είναι **δύναμη του 2**, οπότε δεν χρειάζονται πατώματα/οροφές. Η σχέση γίνεται $T(n) = 2T(n/2) + n$.

Τέχνασμα. Διαιρούμε και τα δύο μέλη με n για να πάρουμε ωραίο τηλεσκοπικό άθροισμα:

$$\frac{T(n)}{n} = \frac{2T(n/2)}{n} + 1 = \frac{T(n/2)}{n/2} + 1$$

Δηλαδή, αν θέσουμε $U(n) = T(n)/n$, τότε $U(n) = U(n/2) + 1$. Εφαρμόζοντας την ίδια ταυτότητα διαδοχικά:

$$\begin{aligned} U(n) &= U(n/2) + 1 \\ &= U(n/4) + 1 + 1 \\ &= U(n/8) + 1 + 1 + 1 \\ &\vdots \\ &= U(n/n) + \underbrace{1 + 1 + \cdots + 1}_{\log_2 n \text{ φορές}} \end{aligned}$$

Όταν $n/n = 1$ έχουμε $U(1) = T(1)/1 = 0$, οπότε:

$$U(n) = \log_2 n \quad \Rightarrow \quad T(n) = n \log_2 n$$

Κλειδί

Αυτή η τεχνική (διαίρεση με n και τηλεσκοπικό άθροισμα) δουλεύει όποτε η σχέση είναι $T(n) = 2T(n/2) + n$. Είναι από τις πιο κομψές αποδείξεις και αξίζει να την ξέρεις απέξω.

15 Απόδειξη #2 — επαγωγή στον διπλασιασμό

Με επαγωγή στο n (πάλι υποθέτοντας $n = 2^k$).

Βάση. $T(1) = 0 = 1 \cdot \log 1$. ✓

Επαγωγική υπόθεση. Έστω $T(n) = n \log n$.

Επαγωγικό βήμα. Θα δείξουμε ότι $T(2n) = 2n \cdot \log(2n)$:

$$\begin{aligned} T(2n) &= 2T(n) + 2n = 2n \log n + 2n = 2n(\log n + 1) \\ &= 2n(\log n + \log 2) = 2n \log(2n) \end{aligned}$$

όπου χρησιμοποιήσαμε $\log 2 = 1$ (με $\log = \log_2$) και $\log a + \log b = \log(ab)$. ■

16 Απόδειξη #3 — πολλαπλασιασμός γραμμών (τηλεσκοπικό)

Ένας ωραίος γενικότερος τρόπος. Για **οποιαδήποτε** σταθερά $c > 0$, αν $T(n) \leq 2T(n/2) + c \cdot n$ τότε $T(n) \leq cn \log n$.

Πώς. Γράφουμε τη σχέση επανειλημμένα, **πολλαπλασιάζοντας κάθε γραμμή με 2^{k-1}** για να εμφανιστεί τηλεσκοπιο:

$$\begin{aligned} T(n) &\leq 2T(n/2) + cn \\ 2T(n/2) &\leq 4T(n/4) + 2c(n/2) = 4T(n/4) + cn \\ 4T(n/4) &\leq 8T(n/8) + 4c(n/4) = 8T(n/8) + cn \\ &\vdots \\ 2^{k-1}T(n/2^{k-1}) &\leq 2^k T(n/2^k) + cn \end{aligned}$$

Προσθέτοντας όλες τις γραμμές, οι μεσαίοι όροι **τηλεσκοπούνται**:

$$T(n) \leq 2^k T(n/2^k) + cn \cdot k$$

Για $n = 2^k$ (οπότε $k = \log_2 n$) και $T(1) = 0$:

$$T(n) \leq 0 + cn \log n = cn \log n$$

Άρα $T(n) = O(n \log n)$. ■

17 Απόδειξη #4 — γενική περίπτωση (ισχυρή επαγωγή, με οροφές)

Οι προηγούμενες αποδείξεις υπέθεσαν $n = 2^k$. Τι γίνεται για γενικό n (όχι δύναμη του 2);

Θεώρημα 1. Αν $T(n) = 0$ για $n = 1$ και $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + n$ για $n > 1$, τότε $T(n) \leq n \lceil \log n \rceil$.

Απόδειξη με ισχυρή επαγωγή στο n .

Βάση ($n = 1$): $T(1) = 0 \leq 1 \cdot \lceil \log 1 \rceil = 0$. ✓

Επαγωγική υπόθεση. Έστω ότι η σχέση ισχύει για κάθε $k = 1, 2, \dots, n-1$.

Επαγωγικό βήμα. Δείχνουμε ότι $T(n) \leq n \lceil \log n \rceil$. Για ευκολία θέτουμε $n_1 = \lfloor n/2 \rfloor$ και $n_2 = \lceil n/2 \rceil$. Είναι $n_1 + n_2 = n$ και $n_1 \leq n_2$.

$$\begin{aligned} T(n) &= T(n_1) + T(n_2) + n \\ &\leq n_1 \lceil \log n_1 \rceil + n_2 \lceil \log n_2 \rceil + n \quad (\text{επαγ. υπόθεση}) \\ &\leq n_1 \lceil \log n_2 \rceil + n_2 \lceil \log n_2 \rceil + n \quad (n_1 \leq n_2) \\ &= n \lceil \log n_2 \rceil + n \\ &\leq n(\lceil \log n \rceil - 1) + n \quad (*) \\ &= n \lceil \log n \rceil \end{aligned}$$

Δικαιολόγηση του (*). Παρατηρούμε:

$$n_2 = \lceil n/2 \rceil \leq \frac{2^{\lceil \log n \rceil}}{2} = 2^{\lceil \log n \rceil - 1}$$

(γιατί $n \leq 2^{\lceil \log n \rceil}$ από τον ορισμό της οροφής). Άρα $\log n_2 \leq \lceil \log n \rceil - 1$, και επειδή το αριστερό μέλος γίνεται ακέραιος όταν παίρνουμε οροφή, $\lceil \log n_2 \rceil \leq \lceil \log n \rceil - 1$. ■

Προσοχή

Γιατί χρειαζόμαστε αυτή την «βαρετή» απόδειξη; Γιατί οι προηγούμενες υπέθεταν $n = 2^k$. Στην πραγματικότητα το n μπορεί να είναι **οτιδήποτε** (π.χ. 100, 1000, 13.547). Η απόδειξη #4 σου

δίνει εγγύηση **για κάθε** n . Στις εξετάσεις, αν δεν λείει διαφορετικά, χρησιμοποίησε την απλούστερη απόδειξη ($n = 2^k$) — είναι αποδεκτή για κάτω/άνω φράγμα.

18 Γιατί στη μέση και όχι « $n - 2$ και 2 »;

Πρέπει να εξηγήσουμε γιατί διαιρούμε σε ίσα μέρη. Αν διαιρούσαμε σε «μεγάλο + μικρό» — π.χ. $T(n) = T(n - 2) + T(2) + n$ — τότε $T(n) = T(n - 2) + \Theta(n)$. Λύνοντας:

$$T(n) = \Theta(n) + \Theta(n - 2) + \Theta(n - 4) + \dots = \Theta(n^2)$$

Δηλαδή χάσαμε όλο το όφελος του D&C — γυρίσαμε στο $\Theta(n^2)$ της bubble sort. Το «στη μέση» είναι κρίσιμο: μόνο τότε το βάθος του δέντρου είναι $\log n$ και όχι n .

19 Γενίκευση: άνισος αλλά αναλογικός διαχωρισμός

Σπόιλερ: το βάθος του δέντρου είναι $\log n$ ακόμα και αν χωρίσουμε σε **οποιαδήποτε σταθερή αναλογία** — όχι μόνο 50/50.

Παράδειγμα

Παράδειγμα. $T(n) = T(n/3) + T(2n/3) + cn$.

Δέντρο αναδρομής. Το αριστερό υποδέντρο σπάει $1/3$ κάθε φορά, το δεξί $2/3$. Το βαθύτερο μονοπάτι (πάντα δεξιά) έχει βάθος $\log_{3/2} n$ — ακόμα **λογαριθμικό**. Κάθε επίπεδο κοστίζει cn ακριβώς όπως πριν (το σύνολο μεγεθών στο επίπεδο είναι πάντα n). Άρα συνολικά $T(n) = O(n \log n)$, ακριβώς όπως η ισόρροπη mergesort.

Κλειδί

Γενικό αξίωμα. Όσο ο διαχωρισμός είναι **αναλογικός** (κάθε υποπρόβλημα είναι **σταθερό κλάσμα** του γονικού), το βάθος είναι $O(\log n)$ και η συνολική δουλειά $O(n \log n)$ — όταν κάθε επίπεδο κοστίζει $O(n)$.

20 Άλλο D&C: ο πύργος του Hanoi

D&C δεν είναι μόνο για ταξινόμηση. Ένα κλασικό παράδειγμα:

Παράδειγμα

Πρόβλημα. Έχουμε 3 στύλους (1, 2, 3) και n δίσκους τοποθετημένους στον στύλο 1 σε **κωνική μορφή** (μεγάλος κάτω, μικρός πάνω). Πρέπει να μεταφέρουμε **όλους τους δίσκους στον στύλο 3**, διατηρώντας την κωνική μορφή.

Κανόνες.

1. Σε κάθε γύρο μετακινούμε **μόνο έναν δίσκο**.
2. Σε κάθε κίνηση αφαιρούμε τον δίσκο που βρίσκεται **στην κορυφή** κάποιου στύλου και τον τοποθετούμε στην κορυφή **κάποιου άλλου** στύλου.
3. **Δεν επιτρέπεται** ένας δίσκος να τοποθετηθεί πάνω σε άλλον με **μικρότερη διάμετρο**.

20.1 D&C λύση

1. **Βάση** ($n = 1$): μία κίνηση — μεταφορά του δίσκου από Στύλο 1 $\rightarrow$ Στύλο 3.
2. **Επαγωγή**. Έστω ότι ξέρουμε να λύνουμε για $n - 1$ δίσκους. Για n δίσκους: μετάφερε τους $n - 1$ πάνω δίσκους από Στύλο 1 $\rightarrow$ Στύλο 2 (αναδρομική κλήση, με Στύλο 3 ως βοηθητικό)· μετάφερε τον n -οστό (μεγαλύτερο) δίσκο από Στύλο 1 $\rightarrow$ Στύλο 3 (μία κίνηση)· μετάφερε τους $n - 1$ δίσκους από Στύλο 2 $\rightarrow$ Στύλο 3 (πάλι αναδρομική κλήση).

Πολυπλοκότητα. Έστω $T(n)$ ο αριθμός κινήσεων:

$$T(n) = T(n - 1) + 1 + T(n - 1) = 2T(n - 1) + 1$$

με $T(1) = 1$. Λύνοντας: $T(n) = 2^n - 1$. (Επαλήθευση: $T(1) = 1$, $T(2) = 2 \cdot 1 + 1 = 3$, $T(3) = 2 \cdot 3 + 1 = 7 = 2^3 - 1$. ✓)

Προσοχή

Η πικρή διαφορά με τη mergesort. Η mergesort έχει $T(n) = O(n \log n)$ — καλή πολυπλοκότητα. Ο πύργος Hanoi έχει $T(n) = 2^n - 1$ — κακή, εκθετική. Παρά το ίδιο σχήμα (D&C), η αναλογία διαχωρισμού δεν είναι ίδια: στο Hanoi κάθε επίπεδο **διπλασιάζεται**, ενώ στη mergesort το συνολικό κόστος ανά επίπεδο μένει n .

Ηθικό δίδαγμα. Το σχήμα D&C από μόνο του δεν εγγυάται γρήγορο αλγόριθμο. Πρέπει επίσης το συνδυαστικό βήμα να είναι «φτηνό» σε σχέση με το μέγεθος των υποπροβλημάτων.

21 To Master Theorem

Είδαμε ότι αναδρομικές σχέσεις της μορφής $T(n) = aT(n/b) + f(n)$ προκύπτουν συνεχώς. Αντί να αποδεικνύουμε κάθε φορά από την αρχή, υπάρχει ένα **γενικό θεώρημα** που τις λύνει.

Κλειδί

Master Theorem (Θεώρημα Κυρίαρχου Όρου). Έστω $a \geq 1$, $b > 1$, $d \geq 0$ σταθερές και $T(n) = a \cdot T(n/b) + O(n^d)$. Τότε:

$$T(n) = \begin{cases} O(n^d) & \text{εάν } d > \log_b a \\ O(n^d \log n) & \text{εάν } d = \log_b a \\ O(n^{\log_b a}) & \text{εάν } d < \log_b a \end{cases}$$

Σε λόγια: συγκρίνεις τον εκθέτη d του «συνδυαστικού βήματος» με τον $\log_b a$ (τον εκθέτη του δέντρου αναδρομής). Όποιος είναι μεγαλύτερος **κυριαρχεί**.

21.1 Διασθητική απόδειξη — δέντρο αναδρομής

Στο k -οστό επίπεδο του δέντρου αναδρομής υπάρχουν a^k υποπροβλήματα, καθένα μεγέθους n/b^k , καθένα με $O((n/b^k)^d)$ συνδυαστική δουλειά. Άρα η συνολική δουλειά στο επίπεδο k :

$$a^k \cdot O\left(\left(\frac{n}{b^k}\right)^d\right) = O(n^d) \cdot \left(\frac{a}{b^d}\right)^k$$

Άθροισμα από επίπεδο 0 ως $\log_b n$:

$$T(n) = O(n^d) \cdot \sum_{k=0}^{\log_b n} \left(\frac{a}{b^d}\right)^k$$

Το τελικό αποτέλεσμα εξαρτάται από τον λόγο $r = a/b^d$:

- $r < 1$ ($d > \log_b a$): η δουλειά «μειώνεται» από επίπεδο σε επίπεδο. **Το πρώτο επίπεδο (ρίζα) κυριαρχεί.** Η γεωμετρική σειρά συγκλίνει $\rightarrow O(n^d)$.
- $r = 1$ ($d = \log_b a$): η δουλειά είναι **ίση σε κάθε επίπεδο** (όπως στη mergesort). $\log n$ επίπεδα $\times n^d \rightarrow O(n^d \log n)$.
- $r > 1$ ($d < \log_b a$): η δουλειά «αυξάνεται» από επίπεδο σε επίπεδο. **Τα φύλλα κυριαρχούν.** Το άθροισμα κυριαρχείται από τον τελευταίο όρο $\rightarrow O(n^{\log_b a})$.

21.2 Πλήρης υπολογισμός

Το βασικό άθροισμα:

$$\sum_{k=0}^{\log_b n} r^k = \begin{cases} O(1) & r < 1 \\ \log_b n + 1 & r = 1 \\ O(r^{\log_b n}) & r > 1 \end{cases}$$

Για $r > 1$: $r^{\log_b n} = (a/b^d)^{\log_b n} = a^{\log_b n} / b^{d \log_b n} = n^{\log_b a} / n^d$. Άρα συνολικά:

$$T(n) = O(n^d) \cdot \sum_{k=0}^{\log_b n} r^k = \begin{cases} O(n^d) \cdot O(1) = O(n^d) & r < 1 \\ O(n^d)(\log_b n + 1) = O(n^d \log n) & r = 1 \\ O(n^d) \cdot O(n^{\log_b a} / n^d) = O(n^{\log_b a}) & r > 1 \end{cases}$$

Που είναι ακριβώς το **Master Theorem**.

22 Εφαρμογές του Master Theorem

Παράδειγμα

Mergesort. $T(n) = 2T(n/2) + n$. Άρα $a = 2$, $b = 2$, $d = 1$. Είναι $\log_b a = \log_2 2 = 1 = d$ — **Case 2:**

$$T(n) = O(n^1 \log n) = O(n \log n) \quad \checkmark$$

Μας δίνει την ίδια απάντηση όπως οι αποδείξεις #1–#4 παραπάνω, σε **μία γραμμή**.

Παράδειγμα

Δυαδική αναζήτηση. $T(n) = T(n/2) + 1$. Άρα $a = 1$, $b = 2$, $d = 0$. Είναι $\log_b a = \log_2 1 = 0 = d$ — **Case 2:**

$$T(n) = O(n^0 \log n) = O(\log n)$$

Παράδειγμα

Karatsuba (θα δούμε στο επόμενο μάθημα). $T(n) = 3T(n/2) + n$. Άρα $a = 3$, $b = 2$, $d = 1$. Είναι $\log_b a = \log_2 3 \approx 1.585 > 1 = d$ — **Case 3:**

$$T(n) = O(n^{\log_2 3}) = O(n^{1.585})$$

Που είναι **καλύτερο** από το ναίει $\Theta(n^2)$ του πολλαπλασιασμού!

Παράδειγμα

Επιλογή — Quickselect. $T(n) = T(n/2) + n$. Άρα $a = 1$, $b = 2$, $d = 1$. Είναι $\log_b a = 0 < 1 = d$
— **Case 1:**

$$T(n) = O(n^1) = O(n)$$

Γραμμική επιλογή του k -στού στοιχείου — θα τη δούμε αργότερα.

Κλειδί

Πρακτικός κανόνας για εξεταστική. Όταν βλέπεις $T(n) = aT(n/b) + n^d$: (1) υπολόγισε $\log_b a$. (2) σύγκρινέ το με d . (3) διάλεξε case και γράψε το αποτέλεσμα. Τρεις γραμμές — πιο γρήγορο από κάθε άλλη μέθοδο όταν εφαρμόζεται.

23 Περιορισμοί του Master Theorem

Το Master Theorem **δεν** καλύπτει κάθε αναδρομική σχέση:

1. **Άνισος διαχωρισμός:** $T(n) = T(n/3) + T(2n/3) + n \rightarrow$ λύνεται με δέντρο αναδρομής, όχι Master.
2. **Μη πολυωνυμικό $f(n)$:** $T(n) = 2T(n/2) + n \log n \rightarrow$ δεν είναι της μορφής n^d . Χρειάζεται γενίκευση (Master Theorem υπό μορφή Cormen).
3. $a < 1$ ή $b \leq 1$: δεν έχει νόημα στο μοντέλο.
4. **Μη ακέραιο μέγεθος υποπροβλημάτων:** πατώματα/οροφές. Η ασυμπτωτική απάντηση μένει ίδια — αλλά η ακριβής σταθερά αλλάζει.

Στις περιπτώσεις όπου το Master Theorem δεν εφαρμόζεται, **επιστρέφουμε στις τεχνικές #1–#4** που είδαμε για τη mergesort.

24 Είναι η mergesort ασυμπτωτικά βέλτιστη;

Στο δεύτερο μάθημα ορίσαμε «ασυμπτωτικά βέλτιστο» αλγόριθμο. Για την ταξινόμηση **με συγκρίσεις**, υπάρχει το γνωστό αποτέλεσμα:

Θεώρημα (κάτω φράγμα). Κάθε αλγόριθμος ταξινόμησης που χρησιμοποιεί **μόνο συγκρίσεις** στοιχείων απαιτεί $\Omega(n \log n)$ συγκρίσεις στη χειρίστη περίπτωση.

Η απόδειξη είναι μέσω **δέντρου απόφασης**: υπάρχουν $n!$ δυνατές μεταθέσεις, άρα το δέντρο έχει $\geq n!$ φύλλα, άρα ύψος $\geq \log_2(n!) = \Theta(n \log n)$.

Συμπέρασμα: mergesort $O(n \log n)$ άνω· κάθε ταξινόμηση με συγκρίσεις $\Omega(n \log n)$ κάτω. Άρα η mergesort είναι **ασυμπτωτικά βέλτιστη** στην οικογένεια αλγορίθμων ταξινόμησης με συγκρίσεις.

Διαίσθηση

Η μόνη μέθοδος να ξεπεράσεις το $\Omega(n \log n)$ είναι να **μην** χρησιμοποιήσεις μόνο συγκρίσεις — π.χ. counting sort / radix sort, που χρησιμοποιούν δομή των δεδομένων (ακέραιοι σε γνωστό εύρος). Αυτοί τρέχουν σε $O(n)$ αλλά λειτουργούν μόνο σε ειδικές περιπτώσεις. Στο μάθημα μένουμε στην ταξινόμηση με συγκρίσεις.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **D&C ως σχήμα:** διαίρει σε ξένα μικρότερα υποπροβλήματα $\rightarrow$ κυρίευσε αναδρομικά $\rightarrow$ συνδύασε. Δικαιολογείται με επαγωγή.
2. **Mergesort:** σπάμε στη μέση, αναδρομικά ταξινομούμε, συγχωνεύουμε με γραμμικό κόστος $\rightarrow O(n \log n)$. **Τέσσερις διαφορετικές αποδείξεις**, κάθε μία χρήσιμη σε διαφορετική εξεταστική.
3. **Γιατί στη μέση:** μόνο αναλογικός διαχωρισμός δίνει $\log n$ βάθος. « $n - 2 + 2$ » κάνει τον αλγόριθμο $\Theta(n^2)$.
4. **Πύργος Hanoi:** D&C με **εκθετική** πολυπλοκότητα ($2^n - 1$). Το σχήμα από μόνο του δεν εγγυάται γρήγορο αλγόριθμο.
5. **Master Theorem:** για $T(n) = aT(n/b) + O(n^d)$ — $d > \log_b a \rightarrow O(n^d)$ · $d = \log_b a \rightarrow O(n^d \log n)$ · $d < \log_b a \rightarrow O(n^{\log_b a})$.
6. **Η mergesort είναι ασυμπτωτικά βέλτιστη** για ταξινόμηση με συγκρίσεις (κάτω φράγμα $\Omega(n \log n)$).

Σημειώσεις από το Algorithms Class Hub — μελετητικός οδηγός για το μάθημα «Αλγόριθμοι και Πολυπλοκότητα (Κ17)», ΕΚΠΑ.